Lecture 5.

Classes, OOP & the Python Object Model.

Software Development & Python
Ivan Solomatin
MIPT

Outline

- Classes and instances
- The Python Object Model
  - Types
  - Attribute lookup
  - Methods
- OOP (Object Oriented Programming):
  - Encapsulation
  - Inheritance
  - Polymorphism
- Special methods and protocols
- Composition, dataclasses, and design choices

Classes and instances

Classes and instances

Why classes?

Functions and dictionaries can already represent almost anything

Data + separate functions

```
user = {
    "name": "Alice",
    "balance": 100,
}

def deposit(user, amount):
    user["balance"] += amount
```

What belongs together?

state

operations on that state

rules / invariants

a public interface

A class packages state and behavior behind one interface.

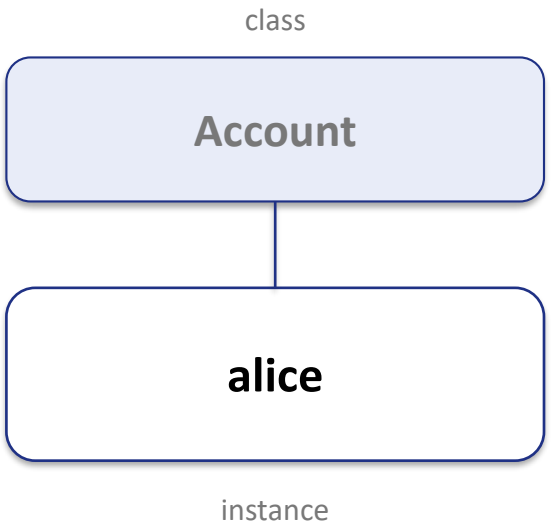
Classes and instances

Our first class

```
class Account: # This is a class
    def __init__(self, balance):
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

# This is an instance of the class
alice = Account(100) #(the object)
alice.deposit(50)
print(alice.balance)
```



A class defines a type of object. Calling the class creates an instance.

Classes and instances

Each instance has its own state

```
alice = Account(100)
bob   = Account(20)

alice.deposit(50)

print(alice.balance) # 150
print(bob.balance)   # 20
```

alice
balance = 150

bob
balance = 20

Python objects can expose named attributes through obj.attribute:

```
alice.balance = 100
alice.owner = "Alice"
alice.currency = "USD
```

alice
balance 100
owner "Alice"
currency "USD"

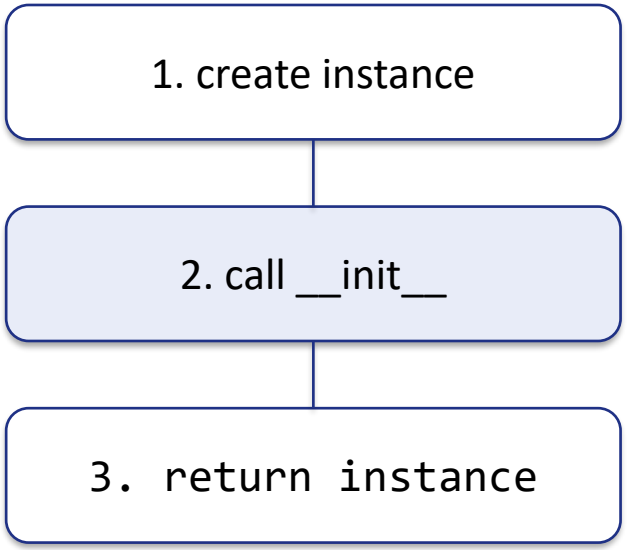
Classes and instances

What does `__init__` do?

Initialization, not object allocation

```
class Account:
    def __init__(self, balance):
        self.balance = balance

alice = Account(100)
```



__init__ configures an instance that has already been created.

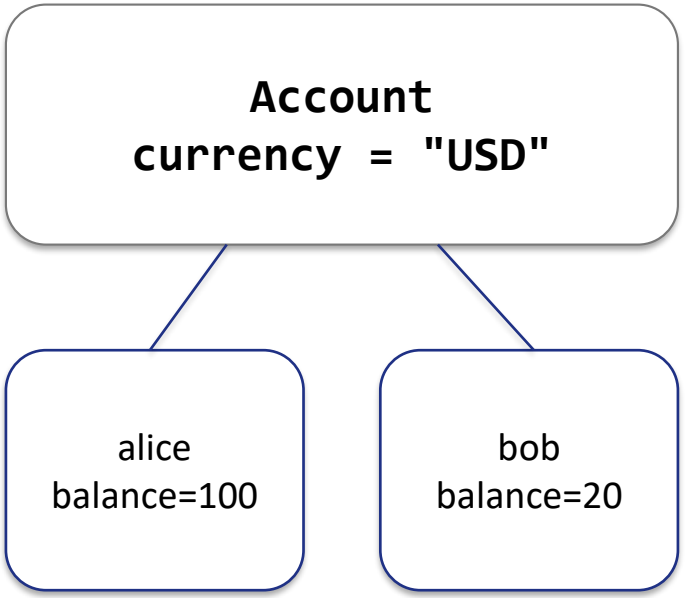
Classes and instances

Instance attributes vs class attributes

```
class Account:
    # class attribute
    currency = "USD"

    def __init__(self, balance):
        # instance attribute
        self.balance = balance

    def print_balance(self):
        # we may use them together
        print(self.balance,
              self.currency)
```



Class attributes are shared through the class; instance attributes belong to each instance.

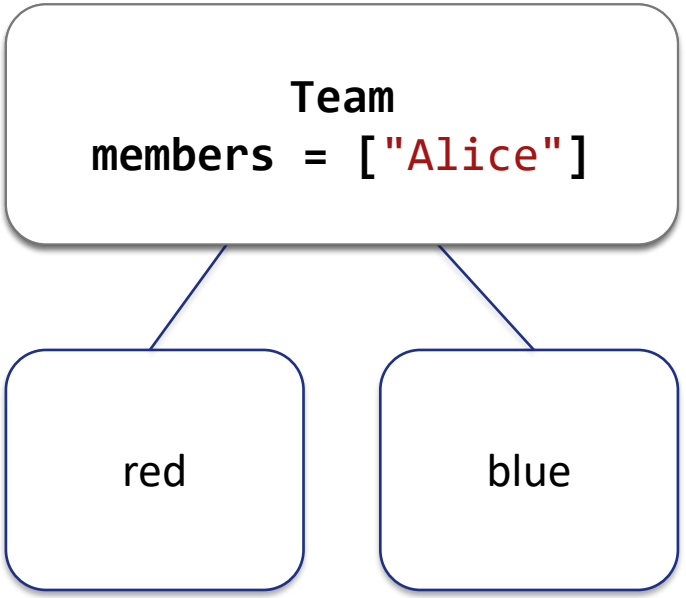
Classes and instances

A common class-attribute trap

```
class Team:
    members = []

red = Team()
blue = Team()

red.members.append("Alice")
print(blue.members)
```



Both instances find the same list.

Mutable class attributes are shared unless shadowed by an instance attribute.

Classes and instances

Look inside: __dict__

```
alice = Account(100)

print(alice.__dict__)
# {'balance': 100}

print(Account.__dict__.keys())
# ... 'currency', '__init__’,
# 'deposit', ...
```

alice.__dict__
instance attributes

Account.__dict__
class attributes + methods

A useful mental model: objects and classes have namespaces mapping names to values.

The Python Object Model

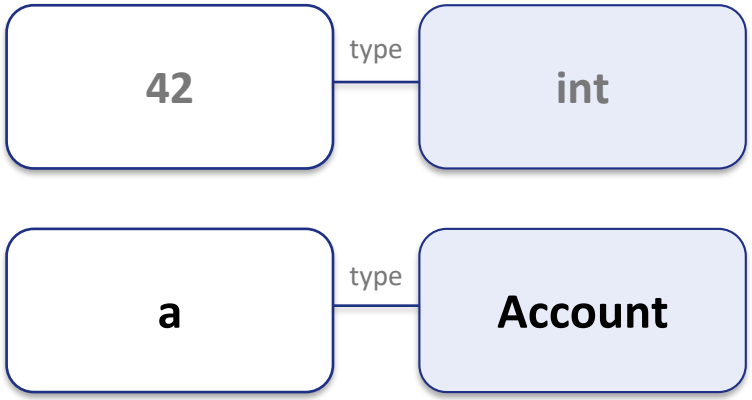
The Python Object Model

Types

Every object has a type

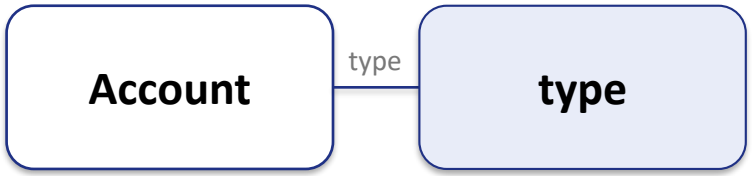
```
type(42)         # int
type("hello")    # str
type([1, 2, 3]) # list

a = Account(100)
print(type(a)) # Account
```



Classes are objects too:

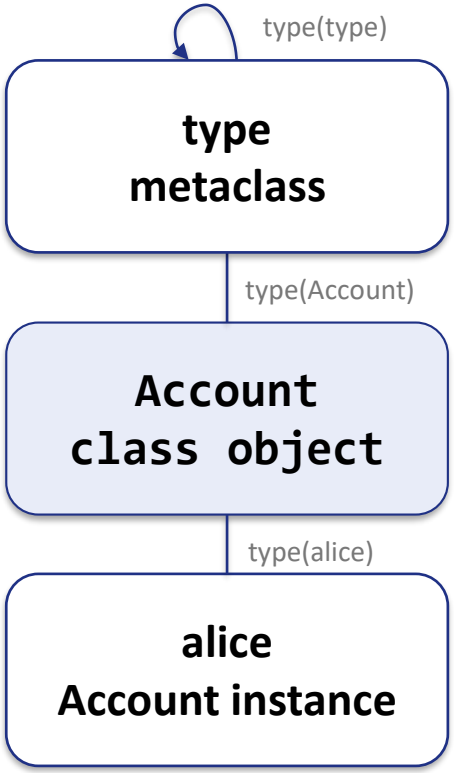
```
print(type(Account)) # type
print(type(type(a))) # type
# <class 'type'>
```



An object carries identity, type, and state/value. Its type determines much of its behavior.

The Python Object Model

Types



The object model is recursive: classes participate in the same world of objects.

The Python Object Model

Attribute lookup

The Python Object Model

Attribute lookup

What does obj.attribute mean?

```
class Account:
    # class attribute
    currency = "USD"

    def __init__(self, balance):
        # instance attribute
        self.balance = balance

    def print_balance(self):
        # we may use them together
        print(self.balance,
              self.currency)
```

Where does Python look?

1. instance

2. class

3. base classes ...

Attribute access is a lookup process, not just “read a field from memory”.

The Python Object Model

Attribute lookup

```
class Account:
    # class attribute
    currency = "USD"

    def __init__(self, balance):
        # instance attribute
        self.balance = balance

    def print_balance(self):
        # we may use them together
        print(self.balance,
              self.currency)
```

alice.balance

found in alice.__dict__

alice.currency

not found in alice.__dict__ →
found in Account.__dict__

Instance state can override names supplied by the class.

The Python Object Model

Attribute lookup

```
class Account:
    currency = "USD"

    def __init__(self, balance):
        self.balance = balance

    def print_balance(self):
        print(self.balance,
              self.currency)
```

```
alice = Account(100)
bob   = Account(200)
alice.print_balance() # 100 USD
bob.print_balance()   # 200 USD

alice.currency = "RUB"
alice.print_balance() # 100 RUB
bob.print_balance()   # 200 USD
```

```
alice = Account(100)
bob   = Account(200)
alice.print_balance() # 100 USD
bob.print_balance()   # 200 USD

Account.currency = "RUB"
alice.print_balance() # 100 RUB
bob.print_balance()   # 200 RUB
```

The Python Object Model

Methods

The Python Object Model

Methods

```
class Account:
    currency = "USD"

    def __init__(self, balance):
        self.balance = balance

    def print_balance(self):
        print(self.balance,
              self.currency)

alice = Account(100)
```

alice.__dict__

{"balance": 100}

Account.__dict__

{'currency': 'USD',
"print_balance": <function ...>,
...}

print_balance is stored on the class.

A normal method is fundamentally a function stored in the class namespace.

The Python Object Model

Methods

Then why does alice.deposit know alice?

```
print(Account.print_balance)
# <function Account.print_balance ...>

print(alice.print_balance)
# <bound method Account.print_balance of
```

Account.print_balance

function

alice.print_balance

bound method

Accessing a function through an instance produces a bound method.

The Python Object Model

Methods

```
def print_balance(self):
    print(self.balance,
          self.currency)
```

alice.print_balance()

self = alice

approximately

Account.print_balance(alice)

Is self a keyword?

```
class Weird:
    def hello(this_object):
        print(this_object)

x = Weird()
x.hello()
```

Valid Python

self is a convention

tools and readers expect it

use **self** in real code

self is a conventional parameter name, not a reserved word.

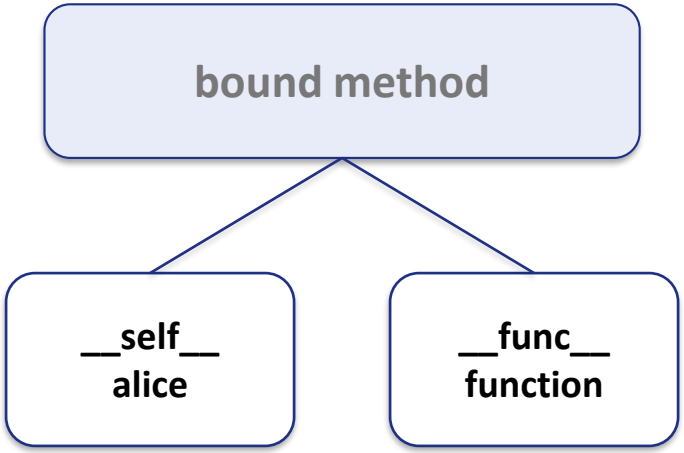
The Python Object Model

Methods

A bound method contains two pieces

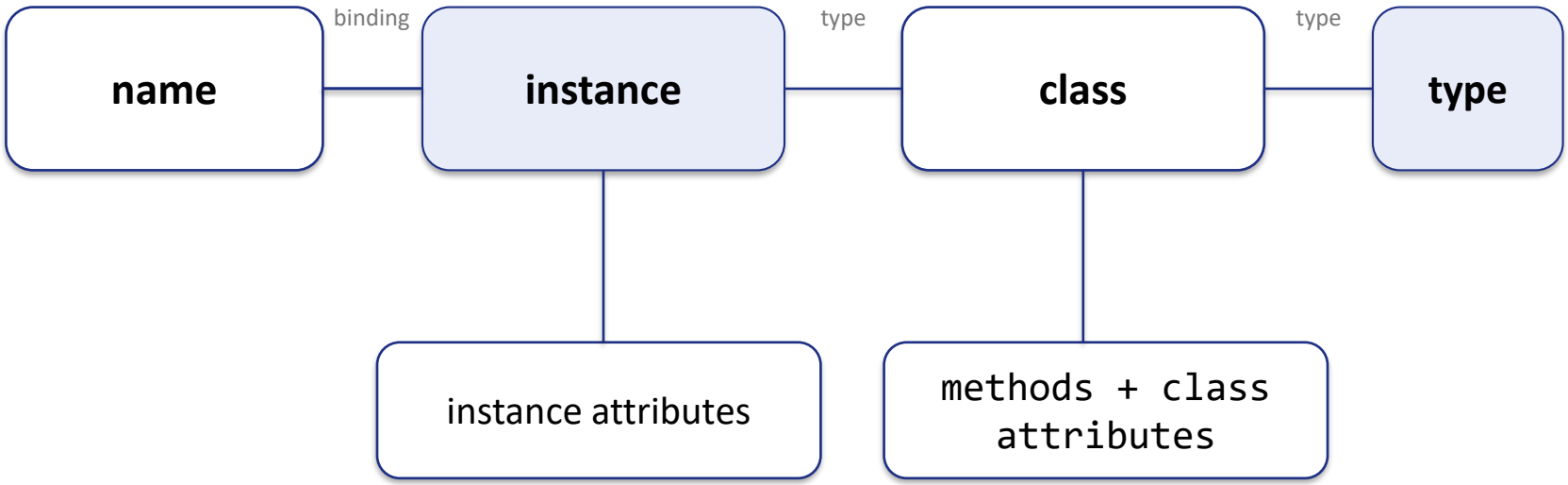
```
m = alice.print_balance

m.__self__   # is alice
m.__func__   # is Account.deposit
```



Python dynamically combines a function with an instance when you access the method.

The Python Object Model



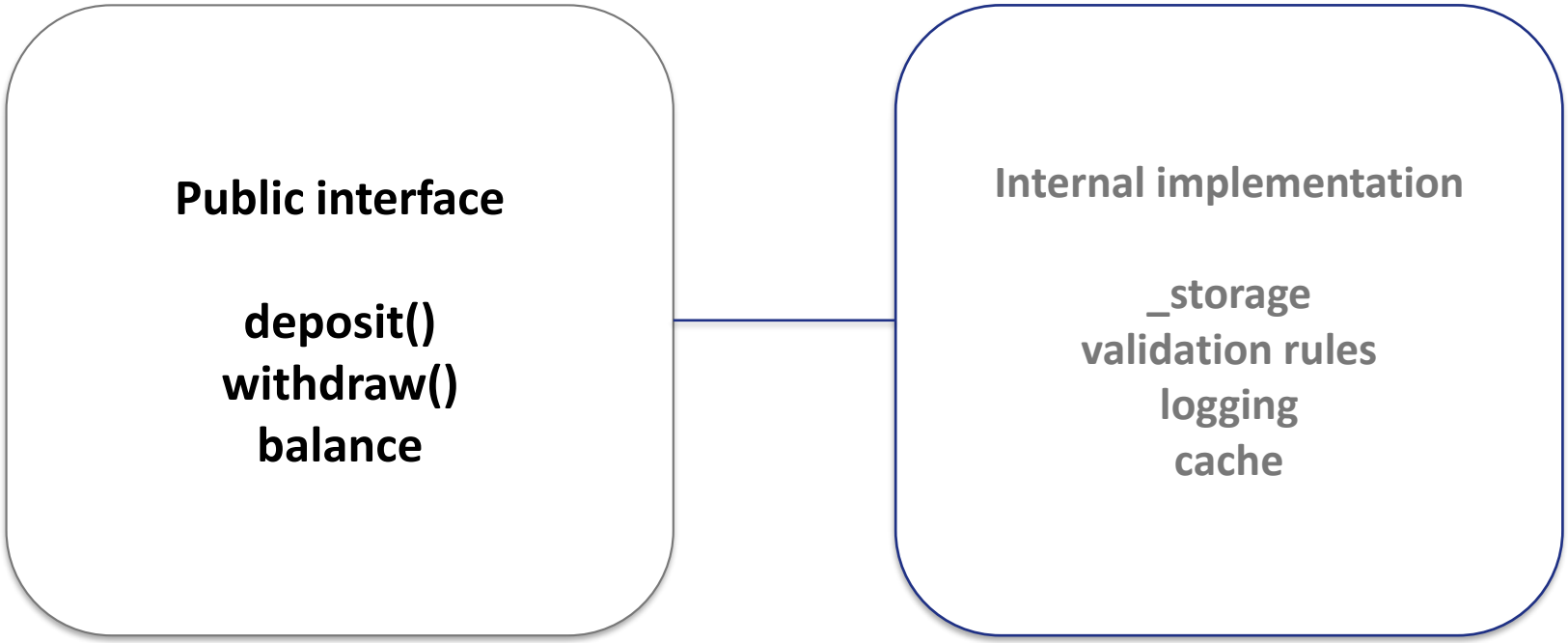
Objects, classes, functions, and methods all participate in one coherent object model.

Object Oriented Programming

- **Encapsulation**
- Inheritance
- Polymorphism

Encapsulation

Encapsulation



Encapsulation lets implementation change while users depend on a stable interface.

Encapsulation

Python has conventions, not Java-style private fields

```
class Account:
    def __init__(self, balance):
        self._balance = balance

# still accessible
alice._balance
```

name → public

_name → internal-use convention

__name → name mangling, not true privacy

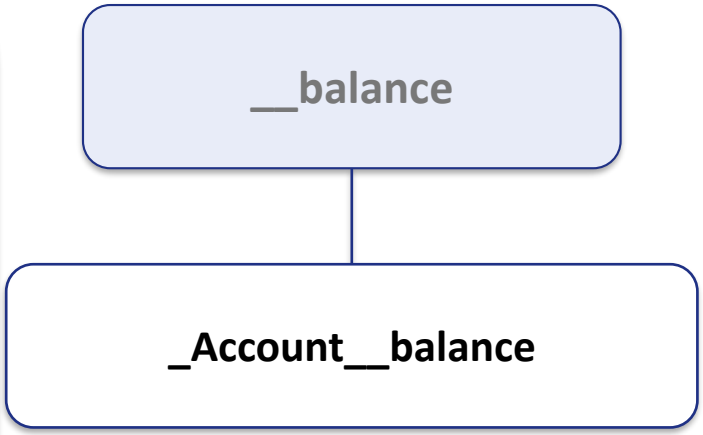
"We are all consenting adults here": Python relies heavily on conventions and interfaces.

Encapsulation

Double underscore: name mangling

```
class Account:
    def __init__(self):
        self.__balance = 100

x = Account()
print(x.__dict__)
# {'_Account__balance': 100}
```



Main goal: avoid accidental name collisions in subclasses.

Name mangling discourages accidental access; it does not provide security.

Encapsulation

A public attribute can become a problem

account.balance = -1_000_000

We need rules

validate values

compute on demand

keep the same public API

Can we keep account.balance syntax while adding behavior behind it?

Encapsulation

Properties: attribute syntax, method behavior

```
class Account:
    def __init__(self, balance):
        self._balance = balance

    @property
    def balance(self):
        return self._balance

    @balance.setter
    def balance(self, value):
        if value < 0:
            raise ValueError("balance<0")
        self._balance = value

a = Account(10)
a.balance = 20
a.balance = -10 → ERROR
```

account.balance

looks like data

getter / setter

runs code

Properties preserve a simple interface while hiding implementation details.

Encapsulation

Encapsulation is about dependency direction



Client code should depend on what an object promises, not how it stores data.

Good encapsulation reduces the number of things the rest of the program needs to know.

Object Oriented Programming

- Encapsulation
- **Inheritance**
- Polymorphism

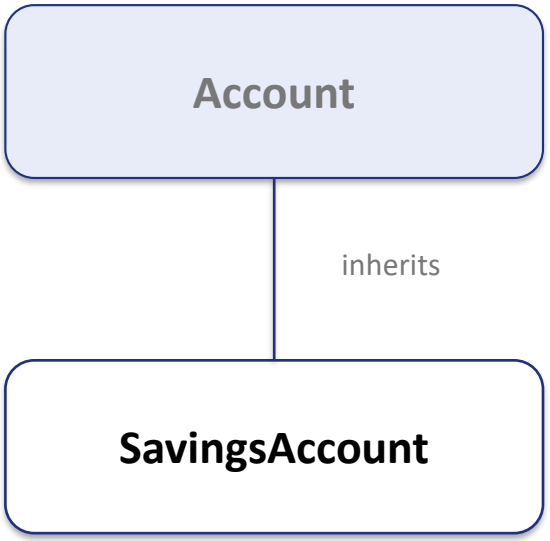
Inheritance

Inheritance

One class extends another

```
class Account:
    def deposit(self, amount):
        ...

class SavingsAccount(Account):
    def add_interest(self):
        ...
```



A subclass starts with the behavior of its base class and may extend or replace it.

Inheritance

Inherited methods are found through lookup

```
class Account:
    def deposit(self, amount):
        ...

class SavingsAccount(Account):
    def add_interest(self):
        ...

s = SavingsAccount()
s.deposit(100)
```

look for deposit

1. instance s

2. SavingsAccount

3. Account ✓

Inheritance plugs directly into the same attribute lookup model.

Inheritance

Overriding

```
class Account:
    def fee(self):
        return 10

class PremiumAccount(Account):
    def fee(self):
        return 0
```

Account().fee()

10

PremiumAccount().fee()

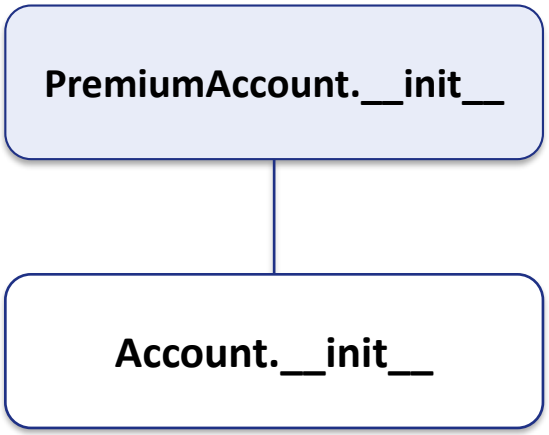
0

A subclass may replace inherited behavior with a more specific implementation.

Inheritance

super(): continue method resolution

```
class PremiumAccount(Account):
    def __init__(self, balance,
                 manager):
        super().__init__(balance)
        self.manager = manager
```



super() is not “call my parent by name”.

super() follows the class method-resolution order (MRO).

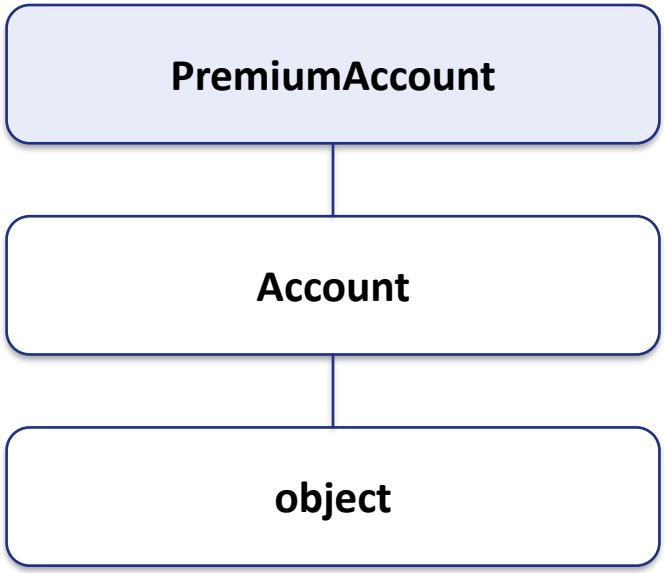
Inheritance

Method Resolution Order (MRO)

Where Python searches for class attributes and methods?

```
PremiumAccount.__mro__

# (PremiumAccount,
#  Account,
#  object)
```

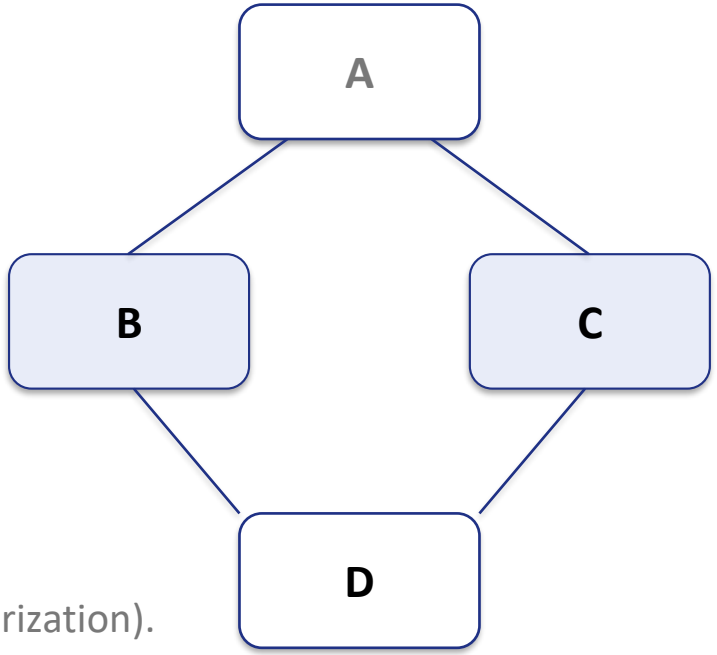


For simple inheritance, the MRO looks intuitive. It matters more with multiple inheritance.

Inheritance

Multiple inheritance exists — but use it deliberately

```
class A: ...
class B(A): ...
class C(A): ...
class D(B, C): ...
```



Python computes a deterministic MRO (C3 linearization).

Know that multiple inheritance exists;
avoid deep hierarchies unless the design clearly benefits.

Object Oriented Programming

- Encapsulation
- Inheritance
- **Polymorphism**

Polymorphism

Same operation, different behavior

Polymorphism

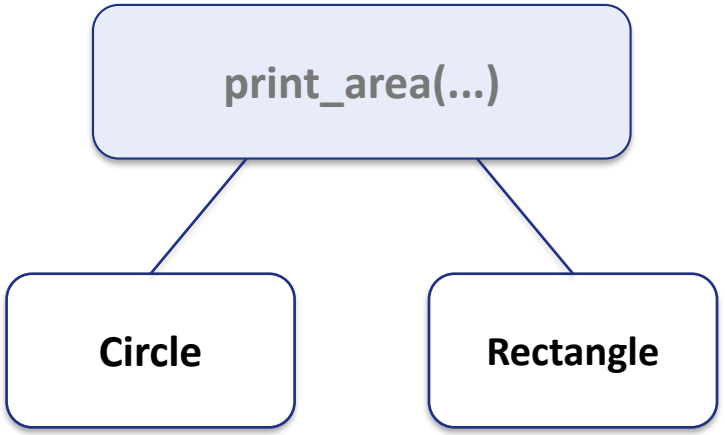
Polymorphism through overriding

```
class Shape:
    def draw(self): ...

class Circle(Shape):
    def draw(self): ...

class Rectangle(Shape):
    def draw(self): ...

for s in shapes:
    s.draw()
```



The caller uses one interface; the actual object determines the behavior.

Polymorphism

Python does not require a shared base class

```
class Circle:
    def draw(self): ...

class Line:
    def draw(self): ...

class MyShape:
    def draw(self): ...

for s in shapes:
    s.draw()
```

Does it have .draw()?

Yes → it works.

No inheritance check is necessary.

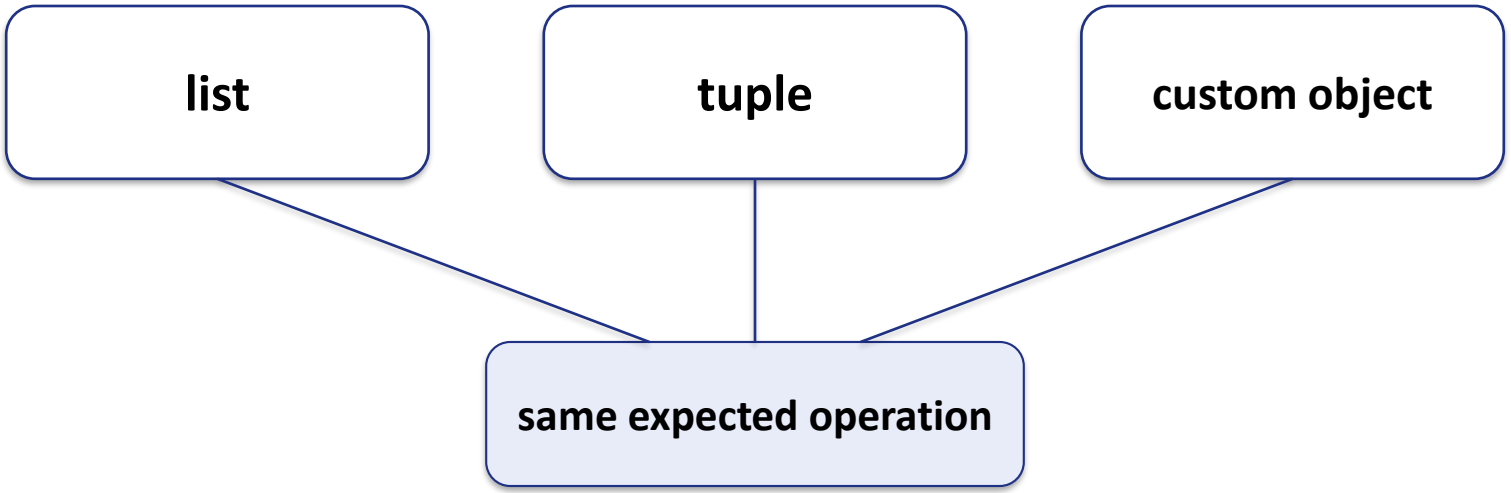
Python usually cares more about supported operations than nominal class hierarchy.

Polymorphism

Duck typing

“If it walks like a duck and quacks like a duck…”

It is a duck.



Behavior can define compatibility: “supports the protocol” rather than “inherits from X”.

Polymorphism

Example: len()

```
len([1, 2, 3])
len("hello")
len({"a": 1})
```

len(obj)

asks the object for its length

```
class Playlist:
    def __len__(self):
        return 42

len(Playlist())  # 42
```

Special methods let user-defined objects participate in Python language protocols.

Special methods & protocols

Special methods & protocols

Python syntax delegates behavior to objects

"Dunder" methods (Double UNDERscore)

Syntax / function	Protocol method
len(x)	x.__len__()
str(x)	x.__str__()
x == y	x.__eq__(y)
x + y	x.__add__(y)
for v in x	iteration protocol
with x	context manager protocol

"Dunder" methods are hooks that connect your objects to Python syntax and built-ins.

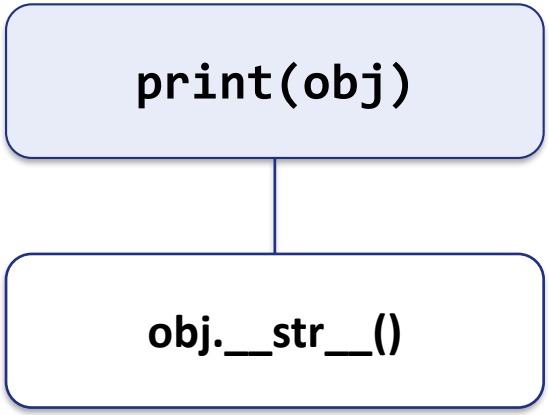
Special methods & protocols

Example: __str__

```
class Account:
  currency = "USD“
  def __init__(self, balance):
    self.balance = balance

  def __str__(self):
    return str(self.balance) + " " + \
           self.currency

print(Account(100)) # 100 USD
```



A language feature can be an ordinary method call hidden behind friendly syntax.

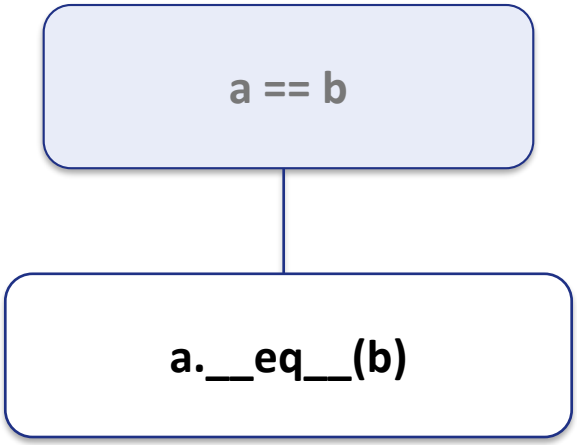
Special methods & protocols

Example: equality is customizable

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y


    def __eq__(self, other):
        return self.x == other.x and \
               self.y == other.y


Point(1, 2) == Point(1, 2)  # True
```



The object model determines the meaning of operators for user-defined types.

Special methods & protocols

Example: operator overloading

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, other):
        return Vector(
            self.x + other.x,
            self.y + other.y
        )
```

```
a + b

# approximately

a.__add__(b)
```

Operator overloading is powerful when the operation has a clear domain meaning.

Special methods & protocols

Protocols go beyond operators

for x in obj	with obj	obj()
iteration __iter__ / __next__	context manager __enter__ / __exit__	callable __call__

We will explore these mechanisms later.

Python is built around protocols: objects opt into behavior by implementing expected methods.

Design choices

Inheritance or composition?

Inheritance: “is a”

```
class SavingsAccount(Account):
    ...
```

Reuse behavior by specialization.

Composition: “has a”

```
class Car:
    def __init__(self):
        self.engine = Engine()
```

Build behavior from cooperating objects.

Prefer composition when the relationship is not genuinely “is-a”.

Design choices

When a class mostly stores data: @dataclass

```python
from dataclasses import dataclass

@dataclass
class Point:
    x: float
    y: float

p = Point(1, 2)
print(p)
# Point(x=1, y=2)
```

@dataclass generates

__init__
__repr__
__eq__
and more, optionally

dataclass removes boilerplate;
it does not change the fundamental object model.

Design choices

Practical design guidance

Question	Useful default
Need state + behavior together?	class
Mostly data?	dataclass
Need “is-a” specialization?	inheritance — carefully
Need to assemble components?	composition
Need compatibility?	small interface / protocol
Need privacy?	encapsulation + conventions

OOP is a design tool, not a requirement that everything must become a class.

Conclusion

What should you remember?

- A class defines behavior shared by its instances.
- Instances and classes both participate in Python's object model.
- Attribute access is lookup through instance, class, and base classes.
- Methods are functions that become bound to an instance.
- Encapsulation separates public interface from implementation.
- Inheritance enables specialization, but composition is often simpler.
- Python polymorphism is often protocol-based: duck typing.
- Special methods connect your objects to Python syntax.

Thank you for watching!